Module 1: Lexing

Regular Languages

Putting the “regular” in “regular expression”.

0.

Assignments

Quiz 1: Java

1.

Grammars

Synonym rolls, just like grammar used to make.

Grammars

Definitions

A **grammar** formally defines which input strings are valid in a **language**.

- **Rules** define **productions** for the language and can use other rules.
- **Terminals** are the raw characters that are part of our language.
- Grammars are <u>not</u> unique - they can define the same languages!

Chomsky Hierarchy

The **Chomsky Hiearchy** is a classification of grammars/languages based on capabilities.

- The classification determines the following:
  - The complexity of the language, effectively what rule combinations are possible.
  - The complexity of the implementation needed to parse/validate the grammar.
- We care about two:
  - Regular; which can be parsed with regex.
  - Context-Free; which is most prog. lang.

Grammars can be recursive!

The relation of recursive rules/terminals affects grammar/language complexity

(e.g. JSON objects & arrays)

```
1. S ::= aS
2. S ::= bX
3. X ::= cX
4. X ::= ε //empty string “epsilon”

X → cX → c
X → cX → ccX → cc
X → cX → ccX → cccX → ccc
X ::= c*

S → bX → bc*
S → aS → abX → abc*
S → aS → aaS → aabX → aabc*
S ::= a*bc*
```

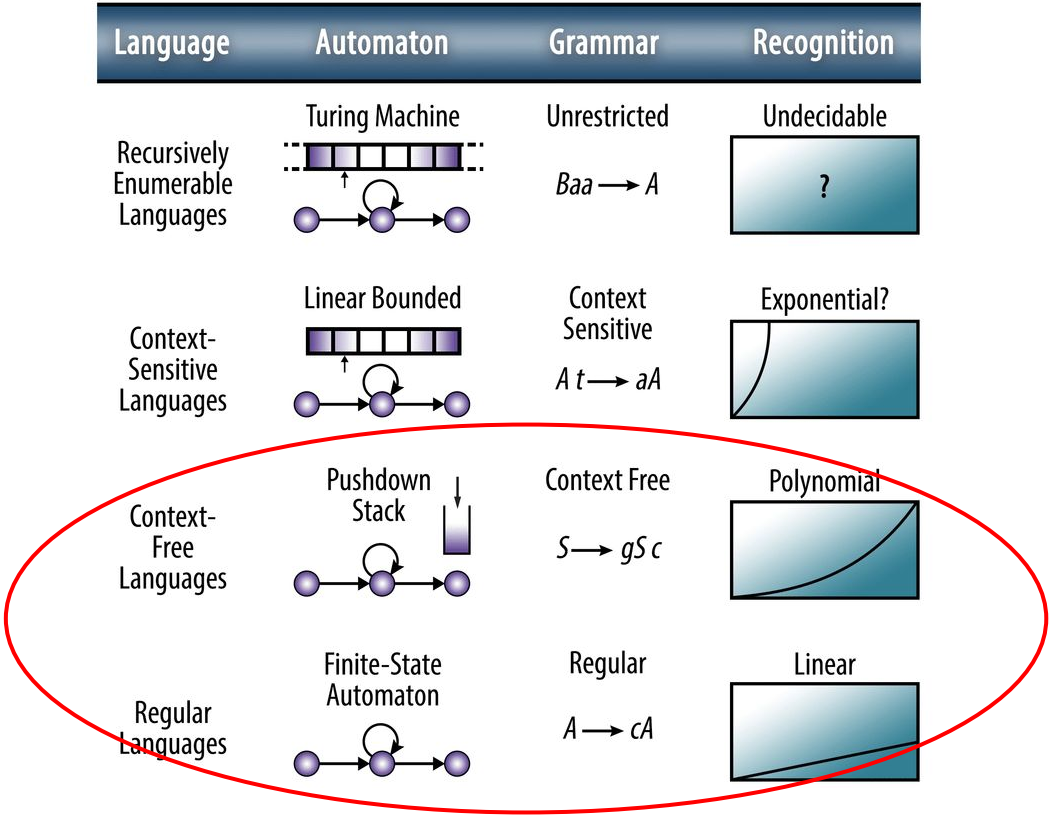
```
1. S ::= SS
2. S ::= (S)
3. S ::= ε

S → (S) → ()
S → (S) → ((S)) → (())
S → (S) → (SS) → (()S) → (()())
S → (S) → (SS) → (()S) → (())
S ::= balanced parenthesis

//can we write a regex for this…?
S ::= \(???\)*
S ::= \(\(???\)*\)*
//this nesting thing seems hard…
```

Example Grammars

2.

Regular Languages

Regular Languages have Regular Grammars.



Chomsky Hierarchy

Regular Language

Definitions

A **regular** grammar/language is:

- A grammar where all terminals are consistently left/right of rules.
  - S ::= aS | bX, X ::= cX | ε
- A language that can be represented using a regular expression.
  - (ab)*
- A language that can be parsed using a finite state machine (FSM).

Advantages/Tradeoffs

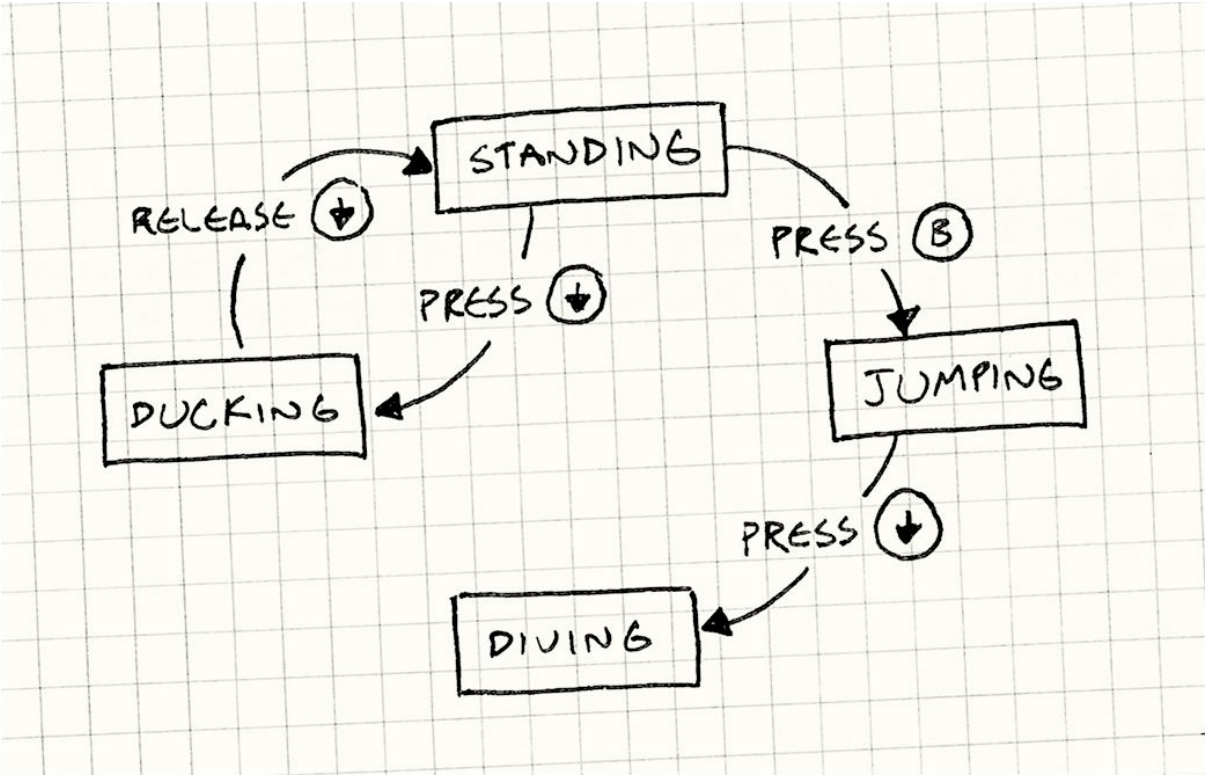
Advantages/Tradeoffs

- Regular languages are great for “simple” input, but not for anything hierarchical.
  - Regex is great for validating form fields…
  - …but definitely not a prog. language.
- FSMs are simple(er) to create and implement.
  - `Pattern.compile` is building a FSM!
- For grammars, FSMs operate in linear time.
  - Also, constant memory (based on rules)

2.

Finite State Automaton

Guest Starring: Finite State Automaton



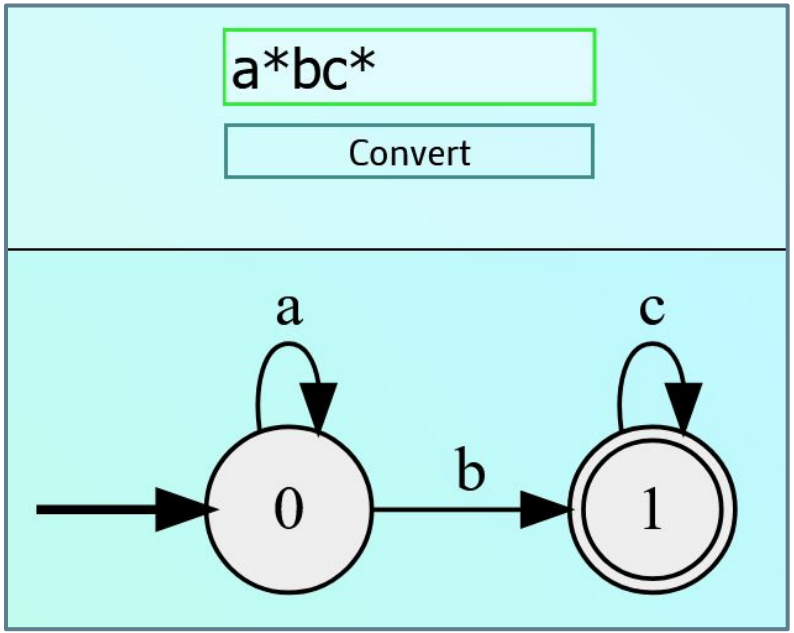
Example Finite State Machine (GPP State)

Finite State Automata (/Machine)

Definitions

A **FSM** has a finite number of **states** and **transitions** between those states.

- A FSM starts in an “inital” state.
- Each step of the input transitions the FSM to the next state.
  - If no valid transition exists, the input is rejected.
- If the input ends on a “final” state, the input is accepted.



Determinism

Deterministic FSM (DFA)

A **DFA** requires all state transitions to be deterministic for the next input.

- Input must transition to one state.
  - Okay: a -> X, b -> X
  - Invalid: a -> X, a -> Y
- Easier model of computation, but requires “optimizing” for regex.

Non-Deterministic FSM (NFA)

A **NFA** allows multiple transitions to be considered simultaneously for the next input.

- Input may transition to multiple states.
  - Valid: a -> X, a -> Y = a -> X,Y
- “Epsilon” (empty) transitions are allowed.
  - Valid: X: ε -> Y, Y: a -> Z, then X: a -> Z
- Harder model of computation (uses a set of states), but better matches regex structure.

DFA == NFA

DFA’s and NFA’s recognize the same languages!
idea: Powerset(NFA) == new DFA

Regex FSM’s

https://27t.github.io/regex-visualizer

Module 1: Lexing

Regular Languages

Putting the “regular” in “regular expression”.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals